CHAPTER 2

Process Management

Process concept · memory layout · states and queues · schedulers · PCB and context switch · fork, exec, wait · threads and models · inter-process communication

QUICK TRICK

Learn the fast tricks first (details below):

1. **Process = program in execution.** Memory layout: **Text, Data, Heap ↑, Stack ↓**. The OS keeps each process's details in its **PCB**.
2. **Five states:** New → Ready → Running → Waiting → Terminated. Running → Ready on **time out**; Running → Waiting on **I/O**; Waiting → **Ready** (never directly to Running).
3. **fork()** returns **0 to the child** and the **child's PID to the parent**. After n fork() calls in a row there are 2^n processes.
4. **Threads of one process share** code, data, heap and files; each thread has its **own** registers, program counter and stack.

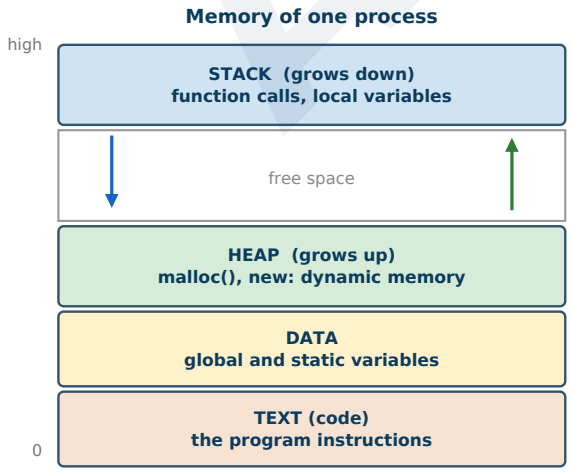
2.1 The process concept

● **LEVEL 1 · BEGINNER**

The problem. A program on disk is just a file of instructions; it does nothing by itself. When you double-click it, something must give it memory, a share of the CPU and access to files. **The idea.** The OS turns the program into a **process**: **a program in execution**. A process is the unit of work in a modern system, and the OS creates, schedules and ends processes all the time.

Analogy. A program is a **recipe** in a book (passive). A process is the **cook actually cooking** with it: using the stove (CPU), counter space (memory) and ingredients (files). Two cooks using the same recipe are two processes of one program.

Point	Program	Process
Nature	**Passive**: a file of instructions on disk	**Active**: a running instance with a state
Lifetime	Exists until deleted	Exists from creation to termination
Resources	None (just storage)	CPU time, memory, files, I/O devices
Count	One program	Many processes can run the same program (two browser windows)
Identity	File name	Process ID (PID)



A process also has (kept in its PCB):

- Process ID (PID) and parent ID
- State: new, ready, running, waiting, terminated
- Program counter (next instruction)
- CPU register values
- Priority and scheduling information
- Memory limits, page tables
- List of open files and devices
- Accounting data (CPU time used)

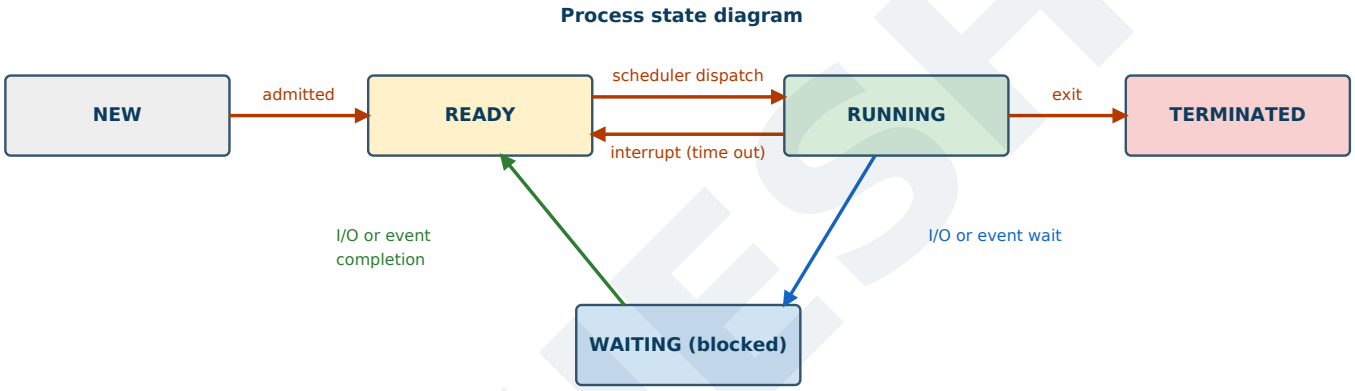
Section	Holds	Size
Text (code)	Program instructions; read-only, shareable	Fixed
Data	Global and static variables	Fixed (initialised and uninitialised parts)
Heap	Memory allocated at run time (malloc, new)	Grows and shrinks
Stack	Function parameters, return addresses, local variables	Grows and shrinks with function calls

BPSC POINT
The heap grows towards **higher** addresses and the stack grows towards **lower** addresses, so they grow towards each other through the free space. If they meet, the process runs out of memory (stack overflow or heap exhaustion).

2.2 Process states and scheduling queues

● **LEVEL 1 · BEGINNER**

A process changes its **state** as it runs. At any moment, on a single CPU, **only one process is Running**; many may be Ready or Waiting.



State	Meaning
New	The process is being created
Ready	Loaded in memory and **waiting for the CPU**
Running	Its instructions are being executed on the CPU
Waiting (blocked)	Waiting for an event: I/O completion, a signal, a resource
Terminated	Finished; the OS is cleaning up its resources

Transition	Cause
New → Ready	OS **admits** the process (long-term scheduler)
Ready → Running	**Dispatch**: the CPU scheduler picks it
Running → Ready	**Interrupt**: time slice expired, or a higher-priority process arrived (pre-emption)
Running → Waiting	It requests **I/O** or waits for an event
Waiting → Ready	The I/O or event it waited for is **complete**
Running → Terminated	It calls **exit** or is killed

COMMON MISTAKE
A process in the Waiting state does **not** go straight back to Running. After the I/O completes it goes to **Ready** and must wait to be scheduled again. Also, there is **no** arrow Ready → Waiting.

Suspended states (7-state model). When memory is short, the OS can **swap** a process out to disk. It then becomes **Suspended Ready** or **Suspended Waiting** and returns to memory later.

Scheduling queues

- **Job queue**: all processes in the system. **Ready queue**: processes in memory ready to run (a linked list of PCBs). **Device (I/O) queues**: processes waiting for a particular device (disk queue, printer queue).
- A process moves between these queues during its life; the CPU scheduler picks from the **ready queue**.

Scheduler	Also called	Job	Speed
Long-term	Job scheduler	Picks which jobs from the pool on disk are brought into memory (New → Ready). **Controls the degree of multiprogramming**; should mix I/O-bound and CPU-bound jobs	Slow (seconds to minutes); may be absent in time-sharing systems
Short-term	CPU scheduler	Picks which ready process runs next on the CPU (Ready → Running)	**Very fast** (every few milliseconds)
Medium-term	Swapper	Swaps processes out of and back into memory to reduce the degree of multiprogramming	Medium

Term	Meaning
I/O-bound process	Spends more time doing I/O than computing; many short CPU bursts
CPU-bound process	Spends more time computing; few long CPU bursts
Dispatcher	The module that **gives control of the CPU** to the process chosen by the short-term scheduler: switches context, switches to user mode, jumps to the right place in the program. **Dispatch latency** = time it takes to stop one process and start another

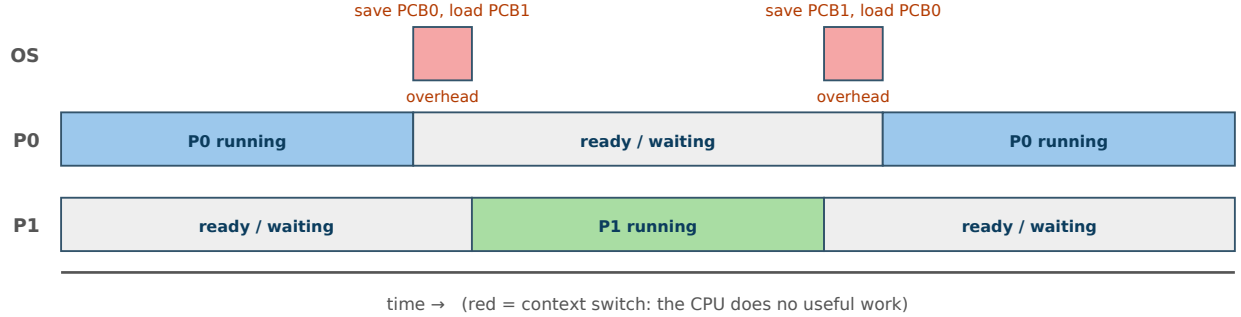
2.3 Process Control Block (PCB) and context switch

● LEVEL 2 · EXAM

The OS keeps all information about a process in a data structure called the **Process Control Block (PCB)**, also called the **task control block**. There is one PCB per process, and it is the **proof that the process exists**. Ready and device queues are lists of PCBs.

PCB field	Contents
Process ID (PID)	Unique number; also parent PID
Process state	New, ready, running, waiting, terminated
Program counter	Address of the next instruction to execute
CPU registers	Accumulator, index registers, stack pointer, flags: saved when the process leaves the CPU
CPU-scheduling information	Priority, pointers to scheduling queues
Memory-management information	Base and limit registers, page tables or segment tables
Accounting information	CPU time used, time limits, job number
I/O status information	List of open files and allocated devices

Context switch. When the CPU moves from one process to another, the OS **saves the state (context) of the old process in its PCB** and **loads the saved state of the new process from its PCB**. The **context** is the program counter, registers and memory-management information.



- **Causes:** time slice expiry, an interrupt, a process waiting for I/O, a higher-priority process becoming ready, a system call.

- A context switch is **pure overhead**: during it the system does **no useful work**. Its cost depends on the hardware (number of registers, memory-management support), usually some microseconds.
- **Process switch is costlier than thread switch**, because memory-management state (page tables, caches) must also change.

Example (overhead). Time quantum = 10 ms and a context switch takes 1 ms. Useful fraction = 10 ÷ (10 + 1) = **90.9%**; overhead = **9.1%**. With quantum = 4 ms and switch = 0.5 ms: useful = 4 ÷ 4.5 = **88.9%**. A very small quantum wastes more time in switching.

> **QUICK TRICK**
> **Context switch = save old PCB + load new PCB.** It is overhead, so the OS wants it to be rare and fast. Hardware with several register sets can make it quicker.

2.4 Operations on processes: creation and termination

● **LEVEL 2 · EXAM**

A process can create other processes. The creator is the **parent**, the new ones are **children**, and together they form a **process tree**. Every process has a PID. In UNIX and Linux the first user process is **init** (now usually **systemd**) with **PID 1**, the ancestor of all others.

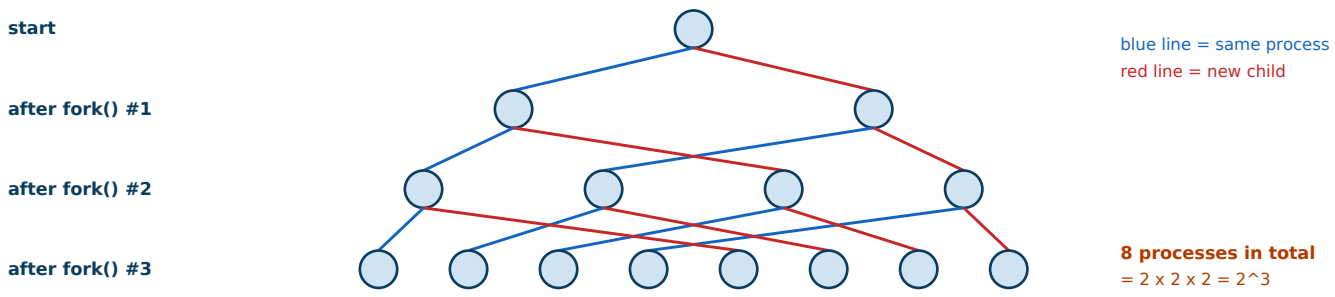
Creation: reasons	Termination: reasons
System start-up A user request (opening a program) A batch job starts A running process creates another (process creation system call)	**Normal exit** (voluntary) **Error exit** (voluntary), for example file not found **Fatal error** (involuntary), for example illegal instruction, divide by zero **Killed by another process** (involuntary)

The fork-exec-wait model of UNIX and Linux

System call	What it does
fork()	Creates a **new child process that is a copy** of the parent (same code, data, open files). Returns **0 in the child**, the **child's PID in the parent**, and **−1 on error**
exec() family	**Replaces** the memory image of the calling process with a **new program**. The PID stays the same; on success exec does not return
wait()	Parent **waits** until a child ends and collects its exit status
exit()	Terminates the process and releases its resources
kill()	Sends a signal to a process (to end it or other purposes)

fork() example (the order of the two lines is not fixed)

```
#include <stdio.h>
#include <unistd.h>
int main() {
    int pid = fork();              /* two processes continue from here */
    if (pid == 0)
        printf("I am the child\n");     /* runs in the child  (pid = 0)  */
    else if (pid > 0)
        printf("I am the parent\n");    /* runs in the parent (pid > 0)  */
    return 0;
}
```



Counting processes. Each fork() splits every existing process into two, so **n successive fork() calls give 2^n processes** ($2^n - 1$ new children). For **fork(); fork(); fork(); printf("Hi");** the output "Hi" appears **2^3 = 8 times** (7

new children). The same holds for a loop **for (i = 0; i < n; i++) fork();**: 2^n processes in all.

With && and ||. In **fork() && fork()** only the parent (non-zero return) runs the second fork, so the total is **3 processes**. In **fork() || fork()** only the child (return 0) runs the second fork, so the total is also **3 processes**.

Special process	Meaning
Zombie	A process that has **terminated** but whose entry stays in the process table because the **parent has not yet called wait()** to read its exit status. It uses no memory, but occupies a PID slot
Orphan	A process whose **parent has terminated first**. It is adopted by **init (PID 1)**, which later collects it
Daemon	A background process that runs without a terminal, providing a service (web server, print spooler)

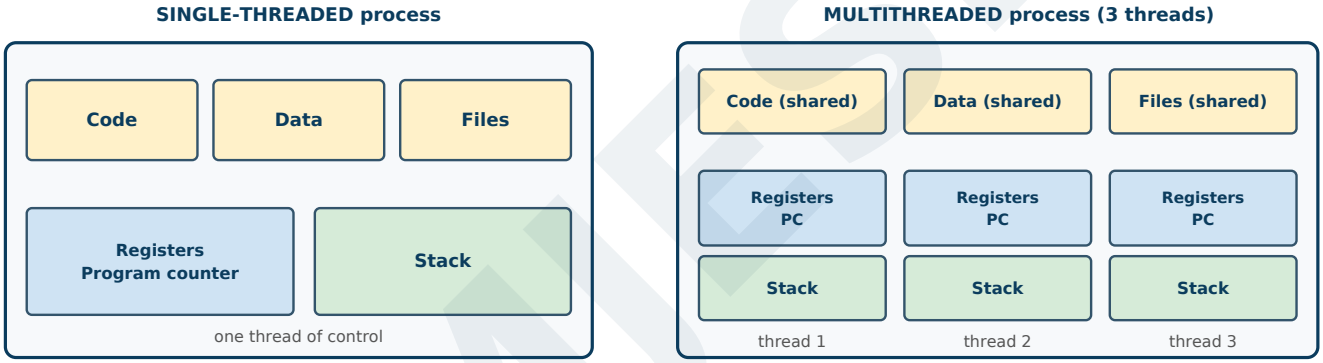
COMMON MISTAKE

- Mixing zombie and orphan: **zombie** = **child** finished, parent has not collected it; **orphan** = **parent** finished first.
- Thinking exec() creates a new process. **fork() creates**, **exec() replaces the program** inside the same process.

2.5 Threads

● **LEVEL 2 · EXAM**

A **thread** is the **basic unit of CPU use**, a lightweight process. A normal (**heavyweight**) process has one thread of control; a **multithreaded** process has several, running in the **same address space**. Examples: a browser (one thread per tab, one for downloads), a word processor (typing, spell-check, auto-save), a web server (one thread per request).



Point	Process	Thread
Weight	Heavyweight	Lightweight
Memory	Own address space	**Shares** code, data, heap and open files with other threads of the same process
Own resources	Everything	Only **thread ID, program counter, registers and stack**
Creation and switching	Slow, costly	**Fast, cheap**
Communication	Needs IPC (pipes, messages)	Direct, through shared memory (but needs synchronisation)
Protection	One process cannot damage another	One faulty thread can crash the whole process

Benefits of multithreading: responsiveness (UI stays alive while another thread works), **resource sharing**, **economy** (cheaper than creating processes), and **scalability** (threads can run in parallel on multiple cores).

	Concurrency	Parallelism
Meaning	Many tasks make progress in overlapping time periods (can be on one core by switching)	Many tasks run **at the same instant** on several cores or CPUs
Needs	Scheduling	Multiple processing units

	User-level threads	Kernel-level threads
Managed by	A thread library in user space; the kernel does **not know** about them	The **operating system kernel**
Speed	Fast to create and switch (no system call)	Slower (system call, kernel work)
Blocking	**One blocking system call blocks the whole process** (all its threads)	Other threads can continue when one blocks
Multicore	**Cannot** use several CPUs at once	Can run on different CPUs in parallel
Examples	Early Java green threads, many-to-one libraries	Windows threads, Linux threads

Multithreading models

Model	Mapping	Good / bad	Example
Many-to-one	Many user threads → **one** kernel thread	Efficient; blocking blocks all; no parallelism	Old green threads
One-to-one	Each user thread → **its own** kernel thread	True parallelism, no blocking problem; creating many threads is costly	**Linux, Windows**
Many-to-many	Many user threads → **a smaller or equal number** of kernel threads	Flexible; complex	Older Solaris, some UNIX

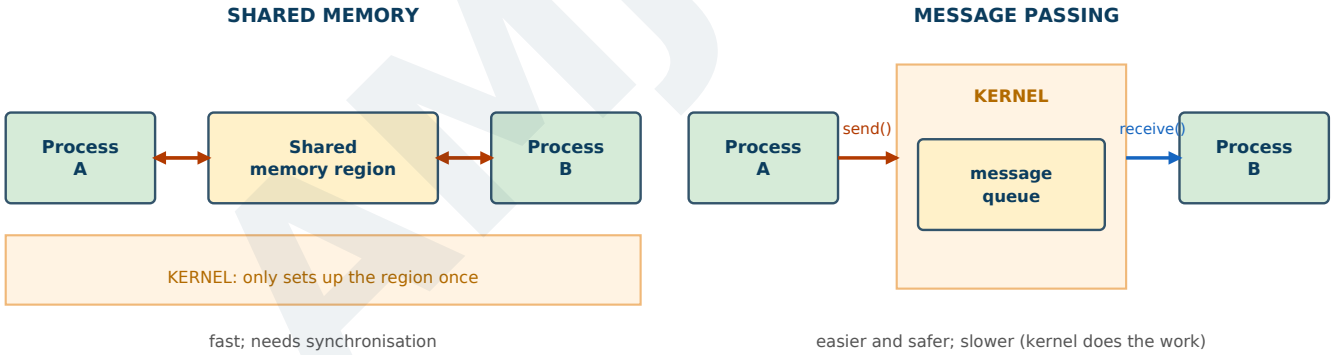
Thread libraries: POSIX Pthreads (UNIX, Linux), **Windows threads**, and **Java threads** (managed by the JVM).

Amdahl's law. If a fraction **S** of a program must run serially and the rest can run in parallel on **N** cores, the best speed-up is **1 ÷ (S + (1 − S) ÷ N)**. **Example:** S = 0.25, N = 4 → 1 ÷ (0.25 + 0.75 ÷ 4) = 1 ÷ 0.4375 = **2.29**. With very many cores the limit is 1 ÷ S = **4**.

2.6 Inter-process communication (IPC)

● LEVEL 2 · EXAM

Processes are either **independent** (do not affect or share data with others) or **cooperating** (affect or are affected by others). Cooperation needs **IPC**, for: **information sharing**, **computation speed-up**, **modularity** and **convenience**.



fast; needs synchronisation

easier and safer; slower (kernel does the work)

Point	Shared memory	Message passing
How	Processes read and write a common memory region	Processes exchange messages with **send()** and **receive()** through the kernel
Speed	**Faster**: kernel only sets it up; later access is ordinary memory access	Slower: every message goes through the kernel
Synchronisation	**Programmer must provide** it (semaphores, locks)	Built in (receive can block)
Best for	Large data on one computer	Small data; **also works across computers** (distributed systems)

IPC mechanism	Idea
Pipe (ordinary / anonymous)	One-way byte channel, usually between a **parent and child**; ls | wc in the shell
Named pipe (FIFO)	A pipe with a name in the file system; works between unrelated processes
Message queue	Kernel-held list of messages that processes add to and take from
Shared memory	Common memory region mapped into several processes (fastest)
Semaphore	Counter used for synchronisation (next chapters)
Socket	Two-way communication endpoint; between processes on the **same or different computers** (IP address + port)
Signal	Short software notification to a process: **SIGINT** (Ctrl+C), **SIGTERM** (polite terminate), **SIGKILL** (kill -9, cannot be caught), SIGSEGV (bad memory access), SIGCHLD (child ended)
RPC	Remote Procedure Call: call a procedure on another computer as if it were local

Message-passing choice	Options
Naming	**Direct** (send to process P) or **indirect** (send to a **mailbox / port**)
Synchronisation	**Blocking (synchronous)**: sender or receiver waits. **Non-blocking (asynchronous)**: continues at once
Buffering	**Zero capacity** (no queue, sender waits), **bounded** (fixed size), **unbounded** (any size)

Producer–consumer problem. A **producer** makes items and puts them in a shared buffer; a **consumer** takes them out. With a **bounded buffer** the producer must wait when it is full and the consumer must wait when it is empty. This classic IPC example needs synchronisation, covered in a later chapter.

Handy commands for processes

Linux	Windows	Meaning
ps, ps aux, top, htop	Task Manager, tasklist	List processes and resource use
kill PID, kill -9 PID	taskkill /PID	End a process
nice, renice	Task Manager: set priority	Change priority
jobs, bg, fg, &	—	Background and foreground jobs; **&** at the end runs a command in the background

2.7 Common mistakes

● LEVEL 2 · EXAM

COMMON MISTAKE

- Saying a program and a process are the same. A program is **passive**; a process is **active**.
- Drawing Waiting → Running or Ready → Waiting in the state diagram. Waiting goes to **Ready**; only Running can go to Waiting.
- Saying fork() returns the child's PID to the child. The **child gets 0**; the **parent gets the child's PID**.
- Forgetting that after 3 fork() calls there are **8** processes, not 6 or 3.
- Confusing zombie (finished child, parent not waiting) with orphan (parent finished first).
- Saying threads have separate memory. They **share** code, data, heap and files; only registers and stack are private.
- Believing user-level threads can use several CPUs. The kernel sees one thread, so they cannot.
- Thinking a context switch does useful work. It is **overhead**.

AMJESH

Quick-Solve Tricks — Summary

Situation	Trick
Process	Program in execution; memory = text, data, heap ↑, stack ↓
States	New → Ready ⇄ Running → Terminated; Running → Waiting → Ready
Schedulers	Long-term (admission, degree of multiprogramming) · Short-term (CPU, fast) · Medium-term (swapping)
PCB	PID, state, PC, registers, scheduling info, memory info, accounting, I/O status
Context switch	Save old PCB, load new PCB; overhead; efficiency = quantum ÷ (quantum + switch time)
fork()	Returns 0 to child, child PID to parent, −1 on error; n forks → 2^n processes
fork() && fork() / fork() || fork()	3 processes in each case
exec()	Replaces the program in the same process (same PID)
Zombie / orphan	Child ended, parent not waiting / parent ended first (adopted by init)
Thread	Shares code, data, heap, files; owns registers, PC, stack
Thread models	Many-to-one (blocks all) · One-to-one (Linux, Windows) · Many-to-many
IPC	Shared memory (fast, needs sync) · Message passing (kernel, slower, also networked) · pipes, sockets, signals
Amdahl	Speed-up = 1 ÷ (S + (1 − S) ÷ N); limit 1 ÷ S

BPSC QUICK CHECK — Chapter 2

Process → active, program → passive	PCB → all information about a process
Running → Ready: time out	Running → Waiting: I/O request
fork() child returns 0	exec() replaces the program
Long-term scheduler → degree of multiprogramming	Short-term scheduler → picks the next process for the CPU
Threads share code, data, files	Zombie → parent has not called wait()

Traps: program vs process · zombie vs orphan · fork return values · process vs thread resources · long-term vs short-term vs medium-term scheduler.

AMJESH

Practice questions

● LEVEL 1 · BEGINNER

- **Q1.** Define a process.
- **Q2.** Name the five states of a process.
- **Q3.** What is a PCB?
- **Q4.** What does fork() return to the child process?
- **Q5.** What is a thread?
- **Q6.** Name two IPC mechanisms.

● LEVEL 2 · EXAM

- **Q7.** Differentiate between a program and a process.
- **Q8.** Which events cause the transitions Running → Ready, Running → Waiting and Waiting → Ready?
- **Q9.** Differentiate between the long-term, short-term and medium-term schedulers.
- **Q10.** What is a context switch? Why is it called overhead?
- **Q11.** Give four differences between a process and a thread.
- **Q12.** Differentiate between a zombie process and an orphan process.

● LEVEL 3 · BPSC / PGT

- **Q13.** A time quantum is 10 ms and a context switch takes 1 ms. Find the useful CPU fraction and the overhead. Repeat for quantum 4 ms and switch 0.5 ms.
- **Q14.** How many times is "Hi" printed by **main() { fork(); fork(); fork(); printf("Hi"); }**? How many new child processes are created? How many processes are created by a loop **for (i = 0; i < 4; i++) fork();**?
- **Q15.** How many processes exist in total after **fork() && fork();** and after **fork() || fork();**?
- **Q16.** In a program 25% of the work is serial. Find the maximum speed-up on 4 cores and on an unlimited number of cores (Amdahl's law).
- **Q17.** Which multithreading model does Linux use? Why can the many-to-one model not use several CPUs, and why does a blocking call in it block all threads?
- **Q18.** Compare shared memory and message passing: which is faster and why, and which suits a distributed system?
- **Q19.** Trace the states and queues of a running process that asks for disk I/O until it runs again.
- **Q20.** Processes spend 60% of their time waiting for I/O. Find the CPU utilisation with 3 processes in memory (utilisation = $1 - p^n$).

AMJESH

Answers

Q	Solution
1	A process is a program in execution; an active entity with its own state, memory, registers and resources.
2	New, Ready, Running, Waiting (blocked), Terminated.
3	The Process Control Block: the OS data structure that stores all information about a process (PID, state, program counter, registers, scheduling, memory and I/O information).
4	0.
5	The basic unit of CPU use, a lightweight process. It has its own thread ID, program counter, registers and stack, and shares code, data and files with other threads of the same process.
6	Pipes, message queues, shared memory, sockets, signals (any two).
7	A program is a passive file of instructions on disk; a process is an active running instance with a state, memory, registers and resources. One program can have many processes.
8	Running → Ready: time slice expires or a higher-priority process arrives (interrupt). Running → Waiting: the process requests I/O or waits for an event. Waiting → Ready: the I/O or event completes.
9	Long-term (job scheduler): admits jobs into memory, controls the degree of multiprogramming, runs rarely. Short-term (CPU scheduler): chooses the next ready process for the CPU, runs very often. Medium-term: swaps processes out and in to reduce the degree of multiprogramming.
10	Saving the state of the running process into its PCB and loading the saved state of the next process from its PCB. It is overhead because the CPU does no useful work for any user process during the switch.
11	Process: own address space, heavyweight, slow creation and switching, needs IPC to communicate. Thread: shares address space with its process, lightweight, fast creation and switching, communicates through shared memory.
12	Zombie: a terminated child whose entry remains because the parent has not called wait(). Orphan: a running process whose parent has terminated; it is adopted by init.
13	10 ÷ 11 = **90.9%** useful, **9.1%** overhead. 4 ÷ 4.5 = **88.9%** useful, **11.1%** overhead.
14	2^3 = **8** times; **7** new children. The loop creates 2^4 = **16** processes in total (15 new children).
15	**3** processes in each case. fork() &&: only the parent runs the second fork. fork() ||: only the child runs the second fork.
16	S = 0.25. N = 4: 1 ÷ (0.25 + 0.75 ÷ 4) = 1 ÷ 0.4375 = **2.29**. N → ∞: 1 ÷ 0.25 = **4**.
17	Linux uses the one-to-one model. In many-to-one all user threads map to a single kernel thread, so only one can run at a time (no multicore use), and when one makes a blocking system call the kernel blocks that single kernel thread, which blocks all threads.
18	Shared memory is faster because after setup the processes access memory directly without the kernel. Message passing suits a distributed system because it works between different computers.
19	Running → the process requests I/O → it goes to the disk's device queue and its state becomes Waiting → the disk finishes (interrupt) → the process moves to the ready queue, state Ready → the scheduler dispatches it, state Running.
20	p = 0.6, n = 3: $1 - 0.6^3 = 1 - 0.216 = 0.784$ = **78.4%**.

REMEMBER

Next: Chapter 3 — CPU Scheduling (scheduling criteria, pre-emptive and non-pre-emptive, FCFS, SJF, SRTF, Priority, Round Robin, multilevel queues, with Gantt-chart numericals for waiting and turnaround time).

Roadmap: Ch 4 Process Synchronisation · Ch 5 Deadlocks · Ch 6 Memory Management · Ch 7 Virtual Memory · Ch 8 File Systems · Ch 9 I/O and Disk Management · Ch 10 Protection, Security, Linux and Windows basics.